

ASSIGNMENT-6.4

NAME:-PRATHAM KOUSHIK

ROLL NO:-2403A52182

Task Description #1:

- Start a Python class named Student with attributes name, roll_number, and marks.

Prompt

GitHub Copilot to complete methods for displaying details and checking if marks are above average.

Expected Outcome #1:

- Completed class with Copilot-generated methods like display_details() and is_passed(), demonstrating use of if-else conditions.

```

class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks

    def display_details(self):
        print(f"Student Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Marks: {self.marks}")

    def is_above_average(self, average_marks):
        return self.marks > average_marks

    def has_passed(self, passing_threshold=46):
        return self.marks >= passing_threshold

# Function to get student data from user
def get_student_data():
    name = input("Enter student name: ")
    roll_number = input("Enter roll number: ")
    marks = float(input("Enter marks: "))
    return Student(name, roll_number, marks)

# Example usage:
# Get data for multiple students
num_students = int(input("Enter the number of students: "))
students = []
total_marks = 0

for _ in range(num_students):
    student = get_student_data()
    students.append(student)
    total_marks += student.marks

# Calculate the average marks
average_marks = total_marks / num_students if num_students > 0 else 0

# Display details and check if marks are above average and if the student passed for e
print("\n--- Student Details, Average Comparison, and Pass/Fail Status ---")
for student in students:
    student.display_details()
    if student.is_above_average(average_marks):
        print("Marks are above average.")
    else:
        print("Marks are not above average.")

    if student.has_passed():
        print("Student has passed.")
    else:
        print("Student has failed.")
    print("-" * 30)

print(f"Average marks for the class: {average_marks:.2f}")

```

Output:

```
Enter the number of students: 4
Enter student name: AnvithSai
Enter roll number: 192
Enter marks: 90
Enter student name: Sriharsha
Enter roll number: 177
Enter marks: 45
Enter student name: Saivarshith
Enter roll number: 189
Enter marks: 92
Enter student name: Sathwika
Enter roll number: 190
Enter marks: 95

--- Student Details, Average Comparison, and Pass/Fail Status ---
Student Name: AnvithSai
Roll Number: 192
Marks: 90.0
Marks are above average.
Student has passed.
-----
Student Name: Sriharsha
Roll Number: 177
Marks: 45.0
Marks are not above average.
...
Marks are above average.
Student has passed.
-----
Average marks for the class: 80.50
```

Task Description #2:

- Write the first two lines of a for loop to iterate through a list of numbers. Use a comment prompt to let Copilot suggest how to calculate and print the square of even numbers only.

Expected Outcome #2:

- A complete loop generated by Copilot with conditional logic (if number % 2 == 0) and appropriate output

```
#write a for loop to iterate through a list of numbers and calculate and print the square of even
numbers = input("Enter a list of numbers (comma-separated): ")
numbers = [int(num) for num in numbers.split(",")]

print("Squares of even numbers:")
for num in numbers:
    if num % 2 == 0:
        print(num ** 2)
```

Output:

```
Enter a list of numbers (comma-separated): 2,5,4,9,8,3,6,4,7,5,6,5
Squares of even numbers:
4
16
64
36
16
36
```

Task Description #3:

- Create a class called BankAccount with attributes account_holder and balance. Use Copilot to complete methods for deposit(), withdraw(), and check for insufficient balance.

Expected Outcome #3:

- Functional class with complete method definitions using if conditions and self attributes. Code should prevent overdrawing

```

class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount}.")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount, threshold=1000):
        if amount > 0:
            if self.balance - amount >= threshold:
                self.balance -= amount
                print(f"Withdrew {amount}.")
            else:
                insufficient_amount = (amount - (self.balance - threshold))
                print(f"Insufficient balance. Withdrawal of {amount} would leave balance below the threshold of ")
        else:
            print("Withdrawal amount must be positive.")

    def check_balance(self):
        print(f"Current balance for {self.account_holder}: {self.balance}")

# Function to get bank account data from user
def get_account_data():
    account_holder = input("Enter account holder name: ")
    initial_balance = float(input("Enter initial balance: "))
    return BankAccount(account_holder, initial_balance)

# Example usage:
# Create a bank account
my_account = get_account_data()

while True:
    action = input("Do you want to (deposit), (withdraw), (check balance), or (quit)? ").lower()

    if action == "deposit":
        amount = float(input("Enter deposit amount: "))
        my_account.deposit(amount)
        my_account.check_balance()
    elif action == "withdraw":
        amount = float(input("Enter withdrawal amount: "))
        my_account.withdraw(amount)
        my_account.check_balance()
    elif action == "check balance":
        my_account.check_balance()
    elif action == "quit":
        print("Exiting.")
        break
    else:
        print("Invalid action. Please choose deposit, withdraw, check balance, or quit.")

```

Output:

```

Enter account holder name: bittu
Enter initial balance: 980
Do you want to (deposit), (withdraw), (check balance), or (quit)? deposit
Enter deposit amount: 800
Deposited 800.0.
Current balance for bittu: 1780.0
Do you want to (deposit), (withdraw), (check balance), or (quit)? withdraw
Enter withdrawal amount: 1000
Insufficient balance. Withdrawal of 1000.0 would leave balance below the thres
Current balance for bittu: 1780.0
Do you want to (deposit), (withdraw), (check balance), or (quit)? check balance
Current balance for bittu: 1780.0
Do you want to (deposit), (withdraw), (check balance), or (quit)? quit
Exiting.

```

Task Description #4:

- Define a list of student dictionaries with keys name and score. Ask Copilot to write a while loop to print the names of students who scored more than 75.

Expected Outcome #4:

- A complete while loop generated by Copilot with proper condition checks and formatted output.

```

# students = [{"name": "Alice", "roll_number": "101", "marks": 85}, {"name": "Bob", "roll_number": "102", "marks": 78}, {"name": "Charlie", "roll_number": "103", "marks": 92}] it is a
students = [{"name": "Alice", "roll_number": "101", "marks": 85}, {"name": "Bob", "roll_number": "102", "marks": 78}, {"name": "Charlie", "roll_number": "103", "marks": 92}]
while True:
    user_input = input("Enter a roll number to check marks (or 'exit' to quit): ")
    if user_input.lower() == 'exit':
        break
    for student in students:
        if student["roll_number"] == user_input:
            if student["marks"] > 75:
                print(f"{student['name']} scored more than 75.")
            else:
                print(f"{student['name']} did not score more than 75.")
            break
    else:
        print("Student not found.")

```

Output:

```

Enter a roll number to check marks (or 'exit' to quit): 101
Alice scored more than 75.
Enter a roll number to check marks (or 'exit' to quit): 102
Bob scored more than 75.
Enter a roll number to check marks (or 'exit' to quit): 56
Student not found.
Enter a roll number to check marks (or 'exit' to quit): exit

```

Task Description #5:

- Begin writing a class ShoppingCart with an empty items list. Prompt Copilot to generate methods to add_item, remove_item, and use a loop to calculate the total bill using conditional discounts.

Expected Outcome #5:

- A fully implemented ShoppingCart class with Copilot-generated loops and if-else statements handling item management and discount logic.

```

#class ShoppingCart:
def __init__(self):
    self.items = []
    # complete this code by doing methods to add_item, and use a loop to calculate the total bill using conditional discounts.

class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_item(self, item_name, item_price):
        self.items.append({"name": item_name, "price": item_price})

    def calculate_total(self):
        total = sum(item["price"] for item in self.items)
        if total > 100:
            total *= 0.9 # Apply 10% discount
        return total

# Create a shopping cart instance
cart = ShoppingCart()

# Get user input to add items
while True:
    item_name = input("Enter item name (or 'done' to finish): ")
    if item_name.lower() == 'done':
        break
    item_price = float(input("Enter item price: "))
    cart.add_item(item_name, item_price)

# Calculate and display the total bill
total_bill = cart.calculate_total()
print(f"Total bill: {total_bill}")

```

Output:

```
Enter item name (or 'done' to finish): pen
Enter item price: 10
Enter item name (or 'done' to finish): book
Enter item price: 40
Enter item name (or 'done' to finish): laptop
Enter item price: 40000
Enter item name (or 'done' to finish): done
Total bill: 36045.0
```